

กลุ่ม 5

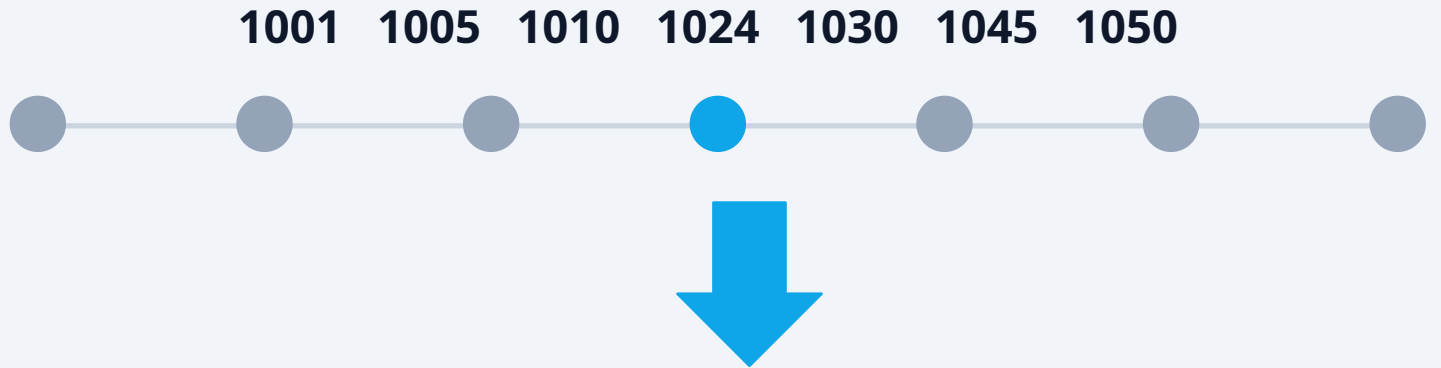
กรณีศึกษาที่ 5

ระบบค้นหาหนังสือ ในห้องสมุด

Binary Search

รายวิชา: การวิเคราะห์ประสิทธิภาพของอัลกอริทึม

อ้างอิงจาก: แบบฝึกหัดกรณีศึกษา หน้า 5 | เนื้อหา: บทที่ 2 Performance Analysis



เริ่มที่ค่ากลาง แล้วตัดช่วงค้นหา “ครึ่งหนึ่ง” ทุกครั้ง

จัดทำเพื่อส่งงานและใช้ฟรีเช่นหน้าชั้น

โจทย์กำหนดให้ทำอะไรบ้าง?

ตรวจให้ครบทั้งคำชี้แจงทั่วไป 1 0 ข้อ + งานเฉพาะกรณีศึกษาที่ 5

1 Input / Output

2 กำหนด n

3 Pseudocode

4 Java

5 Basic Operation

6 จำนวนครั้ง

7 Best / Worst / Average

8 $O / \Omega / \Theta$

9 Auxiliary Space

10 แนวทางปรับปรุง

งานเฉพาะข้อ 5: ออกแบบ Binary Search, เขียน `binarySearchBook(...)`, แสดง low/high/middle และทดลอง 4 กรณี

1-2) Input, Output และความหมายของ n

```
int[] bookIds = {  
    1001, 1005, 1010, 1024,  
    1030, 1045, 1050  
};  
int target = ...;
```

Input

- bookIds: รหัสหนังสือที่เรียงแล้ว
- target: รหัสที่ต้องการค้นหา

Output

- พบ: คืน index
- ไม่พบ: คืน - 1

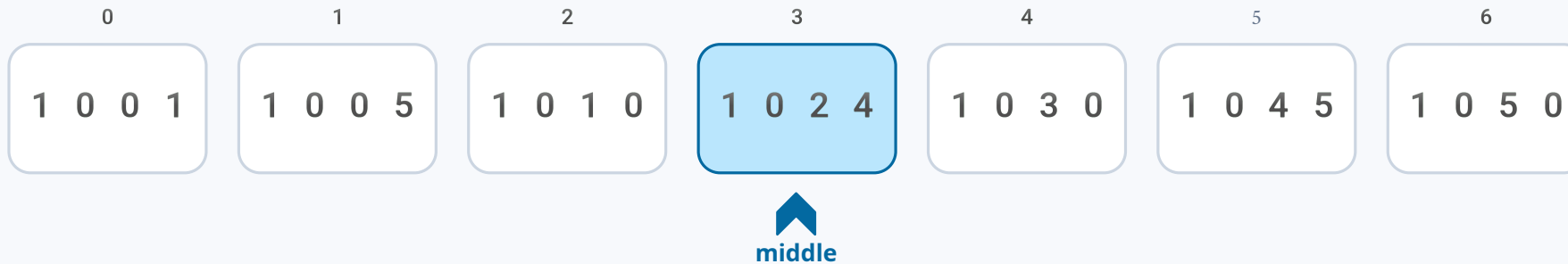
n = จำนวนสมาชิกในอาร์เรย์ bookIds

สำหรับข้อมูลตัวอย่างนี้ $n = 7$

บทที่ 2 หน้า 3: n หมายถึงจำนวน/ขนาดข้อมูล และการค้นหาในอาร์เรย์ใช้จำนวนสมาชิกเป็น n

3) แนวคิดและ Pseudocode ของ Binary Search

เงื่อนไขสำคัญ: ข้อมูลต้องเรียงลำดับก่อน



เริ่มจาก $middle = 3 \rightarrow$ ค่า 1 0 2 4 แล้วเลือกค้นต่อเพียง “ครั้งเดียว”

หลักการ

- 1) เทียบ target กับค่ากลาง
- 2) เท่ากัน $\rightarrow$ พบ
- 3) target มากกว่า $\rightarrow$ ไปครึ่งขวา
- 4) target น้อยกว่า $\rightarrow$ ไปครึ่งซ้าย
- 5) ทำซ้ำจนพบหรือ $low > high$

```
low = 0
high = n - 1
while low <= high
    middle = low + (high-low)/2
    if bookIds[middle] == target return middle
    else if bookIds[middle] < target
        low = middle + 1
    else high = middle - 1
return -1
```

บทที่ 2 หน้า 4: sorted $\rightarrow$ middle $\rightarrow$ eliminate half $\rightarrow$ repeat

4) โปรแกรมภาษา Java: binarySearchBook(...)

```
static int binarySearchBook(int[] bookIds, int target) {  
    int low = 0;  
    int high = bookIds.length - 1;  
  
    while (low <= high) {  
        int middle = low + (high - low) / 2;  
        System.out.println("low=" + low +  
            ", high=" + high + ", middle=" + middle);  
  
        if (bookIds[middle] == target) return middle;  
        else if (bookIds[middle] < target) low = middle + 1;  
        else high = middle - 1;  
    }  
    return -1;  
}
```

ตัวแปรสำคัญ

low = ช้ายสุด
high = ขวาสุด
middle = ตรงกลาง

กรณีพบ

return middle
(คือ index)

กรณีไม่พบ

เมื่อ low > high
return -1

บทที่ 2 หน้า 12: ตัวอย่าง Binary Search ใช้ low = 0 และ high = length - 1

ทดลองตามโจทย์ครบ 4 กรณี

ใช้ bookIds ชุดเดิมจำนวน 7 ค่า

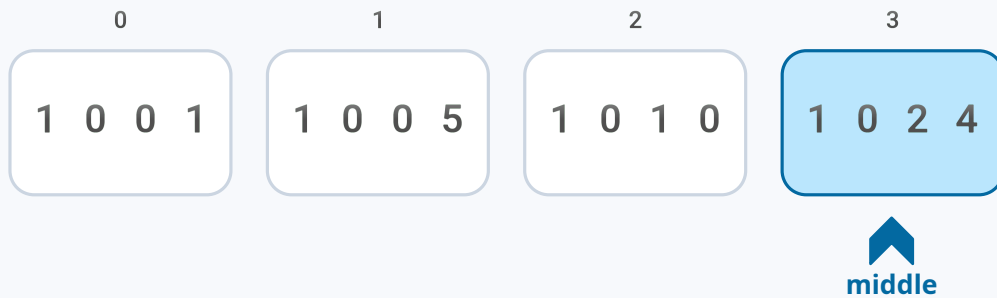
กรณี	target	ผลลัพธ์	จำนวนรอบ
อยู่ตรงกลาง	1 0 2 4	index 3	1
อยู่ตำแหน่งแรก	1 0 0 1	index 0	3
อยู่ตำแหน่งสุดท้าย	1 0 5 0	index 6	3
ไม่มีในอาร์เรย์	1 0 3 5	- 1 (Not found)	3

สังเกต: ค่ากลางเจอกันก็ แต่ค่าที่ปลายหรือค่าที่ไม่มี จะลดช่วงค้นหาไปเรื่อย ๆ จนเหลือช่วงเล็กมาก

รายละเอียดการค้นหา: ตรงกลาง และตำแหน่งแรก

กรณี A: target = 1024

รอบ 1: low=0, high=6, middle=3 → bookIds[3]=1024 → พบ



พบที่ index 3 ใน

กรณี B: target = 1001

รอบ 1 low=0 high=6 middle=3 → 1024
1001 < 1024 → high=2

รอบ 2 low=0 high=2 middle=1 → 1005
1001 < 1005 → high=0

รอบ 3 low=0 high=0 middle=0 → 1001
พบที่ index 0

จำนวนรอบ = 3

รายละเอียดการค้นหา: ตำแหน่งสุดท้าย และไม่พบ

กรณี C: target = 1050

รอบ 1 low=0 high=6 middle=3 → 1024
1050 > 1024 → low=4

รอบ 2 low=4 high=6 middle=5 → 1045
1050 > 1045 → low=6

รอบ 3 low=6 high=6 middle=6 → 1050
พบที่ index 6

กรณี D: target = 1035 (ไม่มี)

รอบ 1 low=0 high=6 middle=3 → 1024
1035 > 1024 → low=4

รอบ 2 low=4 high=6 middle=5 → 1045
1035 < 1045 → high=4

รอบ 3 low=4 high=4 middle=4 → 1030
1035 > 1030 → low=5

ตอนนี้ low=5 > high=4 → return -1

5-6) Basic Operation และจำนวนครั้งที่ทำงาน

Basic Operation (ระดับอัลกอริทึม)

“เปรียบเทียบ target กับ bookIds[middle]”
นับ 1 การตรวจค่ากลางต่อ 1 รอบ

```
if (bookIds[middle] == target)
else if (bookIds[middle] < target)
```

หมายเหตุ: ในโค้ดจริง เมื่อไม่เท่ากัน อาจมีการตรวจ < เพิ่มอีกหนึ่งเงื่อนไข

จำนวนสมาชิกที่ยังเหลือให้ค้นหา



$$n / 2^k \approx 1 \Rightarrow k \approx \log_2 n$$

สำหรับ $n = 7$ ในตัวอย่าง เส้นทางยาวสุดที่ทดลอง = 3 รอบ

7-8) Best / Worst / Average และ Big O, Ω, Θ

กรณี	ลักษณะ	จำนวนรอบโดยระดับการเติบโต	สรุป
Best Case	target อยู่ middle ตั้งแต่รอบแรก	คงที่	$\Theta(1)$
Worst Case	ลดช่วงจนสุดท้าย / ไม่พบ	ลอการิทึม	$\Theta(\log_2 n)$
Average Case	โดยทั่วไปยังลดช่วงครึ่งหนึ่ง	ลอการิทึม	$\Theta(\log_2 n)$

Big O

$O(\log_2 n)$
ขอบเขตบน / worst growth

Omega (Ω)

$\Omega(1)$
ขอบเขตล่าง / best

Theta (Θ)

Best: $\Theta(1)$
Worst/Average: $(\log n)$

ในบทเรียนระบุ Binary Search = $O(\log_2 n)$

9-10) Auxiliary Space และแนวทางปรับปรุง

Auxiliary Space = $O(1)$

เวอร์ชันนี้เป็นแบบ iterative
ใช้ตัวแปรเพิ่มเพียง low, high, middle
จำนวนตัวแปรไม่เพิ่มตาม n

ทำไมไม่ใช่ Recursive?

Recursive จะมี Stack Frame เพิ่มตามจำนวนการเรียก
สำหรับ Binary Search จึงต้องใช้ stack เพิ่มตามความลึก

แนวทางปรับปรุงที่ใช้ในงานนี้

- 1 ใช้ iterative Binary Search เพื่อลดการใช้ Call Stack
- 2 คำนวณ middle ด้วย $low + (high - low) / 2$
- 3 คงข้อมูลให้เรียงลำดับ หากระบบต้องค้นหาซ้ำบ่อย ๆ
- 4 ถ้าข้อมูลยังไม่เรียงและค้นหาเพียงครั้งเดียว อาจใช้ Linear Search แทนการเสียขั้นตอนเรียงก่อน

ตอบคำถามวิเคราะห์ของกรณีศึกษาที่ 5 (ครบ 5 ข้อ)

1. ช่วงค้นหาได้อย่างไร?

ลดเหลือประมาณครึ่งหนึ่งทุกครั้ง: $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$

2. รอบสูงสุดสัมพันธ์กับ $\log_2 n$ อย่างไร?

เพราะแบ่งครึ่งซ้ำ ๆ จนเหลือ 1 ค่า จึงมีจำนวนรอบระดับ $O(\log n)$

3. Best / Worst Case?

Best = $\Theta(1)$ เมื่อเจอกลางทันที; Worst = $(\log n)$ เมื่อค้นถึงจุดสุดท้ายหรือไม่พบ

4. เงื่อนไขสำคัญ?

ข้อมูลต้องเรียงลำดับก่อน จึงตัดสินใจที่ครึ่งซ้าย/ขวาได้ถูกต้อง

5. ถ้ายังไม่เรียง ควรเรียงก่อนค้นครั้งเดียวไหม?

ถ้าค้นครั้งเดียว มักใช้ Linear Search ได้ทันที; ถ้าค้นซ้ำหลายครั้ง การเรียงก่อนอาจคุ้มกว่า (ขึ้นกับจำนวนครั้งและวิธีเรียง)

ข้อ 5 เป็นข้อสรุปเชิงวิเคราะห์จากข้อมูลในบทเรียน: Linear Search ใช้กับข้อมูลไม่เรียงได้ ส่วน Binary Search ต้องเรียง

สรุปคำตอบของกลุ่ม 5

หัวข้อ	คำตอบ
Algorithm	Binary Search
n	จำนวนสมาชิกใน bookIds (ตัวอย่าง n=7)
Basic Operation	ตรวจ target กับ bookIds[middle]
Best Case	$\Theta(1)$
Worst Case	$\Theta(\log_2 n)$
Average Case	$\Theta(\log_2 n)$
Big O	$O(\log_2 n)$
Omega	$\Omega(1)$
Auxiliary Space	$O(1)$ แบบ iterative
เงื่อนไข	ข้อมูลต้องเรียงลำดับ

Binary Search เหมาะกับโจทย์นี้ เพราะข้อมูลเรียงแล้วและลดช่วงค้นหาค้างหนึ่งทุกครั้ง

ภาคผนวก: Java Code แบบเต็ม (นำไปรันได้)

```
public class LibrarySearch {
    static int binarySearchBook(int[] bookIds, int target) {
        int low=0, high=bookIds.length-1;
        while (low <= high) {
            int middle = low + (high-low)/2;
            System.out.println("low="+low+", high="+high+
                               ", middle="+middle+", value="+bookIds[middle]);
            if (bookIds[middle] == target) return middle;
            else if (bookIds[middle] < target) low = middle+1;
            else high = middle-1;
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] bookIds={1001,1005,1010,1024,1030,1045,1050};
        int[] tests={1024,1001,1050,1035};
        for (int target : tests) {
            System.out.println("\nTarget = " + target);
            int result = binarySearchBook(bookIds,target);
            System.out.println(result >= 0 ? "Found index="+result : "Not found (-1)");
        }
    }
}
```

ไฟล์ .java แยกแบบให้ด้วย และตรวจสอบแล้วว่า compile/run ได้